

# Technische Spezifikation: ALDI SÜD Supply Chain Simulationssystem

## 1 Zusammenfassung

---

Die technische Spezifikation definiert ein vollständig implementierbares Simulationssystem der ALDI SÜD Supply Chain, das auf drei Kernkomponenten basiert: einer SQLite-Datenbank mit acht Kerntabellen (events, roles, messages, inventory, kpi\_states, suppliers, escalations, demo\_config), einem Vite+React+TypeScript-Frontend mit fünf Feature-Slices (Simulation, Outlook, Teams, SAP, Manhattan WMS) und MCP-Servern mit JSON-RPC 2.0 und OAuth-2.1-Authentifizierung für die Amazon-Quick-Integration [1, 2, 3].

Das zentrale Designprinzip ist eine agentenbasierte Architektur mit fünf autonomen Rollen-Agenten, die im 10-Sekunden-Takt eine Event-Queue pollen und über schwellenwertbasierte Geschäftsregeln Eskalationsketten auslösen [4]. Der angemeldete Benutzer (Standard-Persona: Markus Weber) als Buyer/Einkäufer verwaltet ca. 54–60 SKUs (3% des Sortiments) und wird über definierte Kanäle erreicht: Teams-Nachrichten bei zeitkritischen Eskalationen (OTD < 90%), Outlook-E-Mails bei formellen Lieferanten-Preisforderungen und SAP-Ariba-Alerts bei Vertragsabweichungen [4]. Bei Rohstoffpreiserhöhungen und Mindestlohnänderungen fungiert er selbst als Erst-Signal-Rolle [4].

Die Spezifikation umfasst acht parametrisierbare Disruptionsszenarien mit Margin-Impacts von 18 bis 125 Basispunkten [4], deren Propagation durch korrelierte Risikocluster modelliert wird [4]. Für den Coding Agent ist die empfohlene Implementierungsreihenfolge klar priorisiert: MCP-Server-Integration zuerst (um frühzeitig zu validieren, dass Amazon Quick den Outlook-MCP als E-Mail-System erkennt), dann Datenbank-Schema mit relativ datierten Seed-Daten, Agent-Engine mit Polling-Loop, REST-API via Soul-Framework und Frontend-Views — wobei alle Schwellenwerte und Szenario-Parameter konfigurierbar gehalten werden sollen, um iterative Anpassungen zu ermöglichen.

## 2 Einleitung und Systemüberblick

---

Diese technische Spezifikation definiert ein vereinfachtes Simulationssystem der ALDI SÜD Supply Chain, das als Grundlage für die Implementierung durch einen Coding Agent dient. Das System simuliert die Interaktion mehrerer Backend-Systeme — SAP (ERP, S/4HANA, Ariba), Microsoft Outlook, Microsoft Teams und Manhattan Solutions (WMS) — und bildet dabei die realen Geschäftsprozesse, Rolleninteraktionen und Risikoantwortmuster von ALDI SÜD ab.

Die Zielarchitektur basiert auf drei Kernkomponenten: einer SQLite-Datenbank als persistente Datenschicht, einem Vite-basierten Webserver für die Frontend-Darstellung der simulierten Systeme, und MCP-Servern (Model Context Protocol) für die Integration mit Amazon Quick. Das Simulationssystem nutzt einen kombinierten Monte-Carlo- und Discrete-Event-Ansatz, der sowohl stochastische Störungsprofile als auch die dynamische Propagation durch ein Fünf-Rollen-Netzwerk abbildet [4]. Die Agenten werden mit Human-in-the-Loop-Orchestrierung modelliert, um realistische Entscheidungsverzögerungen abzubilden [4].

ALDI SÜD Deutschland betreibt ein Netzwerk von über 2.000 Filialen mit einem Umsatz von etwa 20-22 Milliarden Euro und einer operativen Marge von 2-5%. Die Logistikinfrastruktur umfasst 24 regionale Distributionszentren, 5 Frischware-Plattformen und Cross-Docking-Hubs mit multimodalem Transport [5]. Das Sortiment umfasst etwa 1.800-2.000 SKUs, wobei der Fokus auf Eigenmarken liegt [5].

## 3 Rollendefinition Markus Weber (Buyer/Einkäufer)

---

**Dynamische Benutzeridentität.** Die Buyer-Rolle ist nicht auf den Namen „Markus Weber“ festgelegt, sondern wird dynamisch aus dem Login-Namen des angemeldeten Benutzers aufgelöst. Beim Start der Simulation wird der authentifizierte Benutzername (z.B. aus dem Amazon-Quick-Login) in die roles-Tabelle als aktiver Buyer eingetragen. Alle MCP-Server, Agenten und simulierten Systeme adressieren den Benutzer unter diesem Namen — so erkennt Amazon Quick den Benutzer korrekt und routet Nachrichten an die richtige Identität. Die roles-Tabelle enthält ein Feld display\_name, das beim Login dynamisch gesetzt wird und in allen generierten Nachrichten (Outlook, Teams, SAP-Alerts) als Empfängername verwendet wird.

**Persona und Zuständigkeitsbereich.** Der Buyer/Einkäufer (Standard-Persona: Markus Weber) ist in der Holding-Organisation im Buying & Procurement Team angesiedelt. Seine Kernaufgabe besteht darin, das größte Volumen des bestgeeigneten Produkts zu kaufen und die maximale Kosteneinsparung auf nachhaltigste Weise zu liefern [4]. Er verwaltet ein Portfolio von ca. 54-60 SKUs, was etwa 3% des Gesamtassortiments entspricht [5].

**Kategorie-Portfolio.** Für die Simulation wird Markus Weber folgendes Lieferanten-Portfolio zugewiesen: Im Bereich Kaffee verantwortet er die Eigenmarken Markus und Barissimo, die vom Hersteller NewCoffee produziert werden — einem Unternehmen, das in den 1960er Jahren speziell zur Belieferung von ALDI SÜD gegründet wurde [5].

Im Bereich Premium-Schokolade betreut er die Marken Moser Roth und Choceur mit den Herstellern Storck und Ludwig Schokolade.

**KPIs und Entscheidungsbefugnisse.** Die primären KPIs umfassen Cost Savings (%), Supplier Compliance und Payment Terms [4]. Markus Weber eskaliert eigenständig bei Kostenüberschreitung von mehr als 5% gegenüber dem Vertragspreis [4]. Bei einer Bestandsreichweite unter 3 Tagen ohne verfügbaren Alternativlieferanten eskaliert er an die Sortimentsplanung [4].

**Wer kommt auf Markus Weber zu und wann.** Die Logistik-Abteilung kontaktiert ihn bei Lieferproblemen, da er die Lieferantenverträge und Vertragsflexibilität steuert [4]. Dies geschieht insbesondere wenn die On-Time-Delivery unter 90% über eine Woche fällt [4]. Bei Szenarien wie Rohstoffpreiserhöhungen und Mindestlohnänderungen ist Markus Weber die Erst-Signal-Rolle, da er diese über Lieferanten-Preisverhandlungen erkennt [4]. Bei Produktrückrufen wird er direkt vom Lieferanten oder über interne Qualitätskontrolle informiert.

#### 4 Vollständige Rollentaxonomie und Interaktionsmuster

Das Simulationssystem bildet fünf Kernrollen ab, die jeweils als autonomer Agent implementiert werden. Die Einkaufsabteilung operiert als separates Team bei der Holding mit Service-Team- und Procurement-Team-Struktur und konsolidiert Einkaufsvolumina über Länder hinweg [4].

| Rolle                                      | Organisation                            | KPIs                                                                                    | Entscheidungsebene            |
|--------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------|
| Sortimentsplanung / Category Management    | Holding, International Brand Management | SKU-Produktivität, Marge/Kategorie                                                      | Zentral, strategisch          |
| Einkauf / Procurement (Markus Weber)       | Holding, Buying & Procurement Team      | Cost Savings %, Supplier Compliance, Payment Terms                                      | Zentral, taktisch-strategisch |
| Supply Chain Coordinator                   | Holding (innerhalb ISCM)                | Unterstützung der Buying Teams mit Logistik-Know-how                                    | Zentral, koordinierend        |
| Logistik / Distribution                    | ISCM Centre of Excellence               | On-Time-Delivery, Transportkosten/ Einheit, Lagerauslastung, CO <sub>2</sub> -Reduktion | Zentral + Regional            |
| Filialnachbestellung / Store Replenishment | Automatisiert, zentral gesteuert        | OSA, Out-of-Stock-Rate, Bestellgenauigkeit, Schwund                                     | Automatisiert/Regional        |
| Prognose / Demand Forecasting              | In SC-Planungssysteme integriert        | MAPE, Promotions-Uplift-Genauigkeit, Bias-Reduktion                                     | Zentral, systemgestützt       |

Tabelle 1: Rollentaxonomie des ALDI SÜD Supply Chain Simulationssystems

**Sortimentsplanung / Category Management** agiert auf strategischer Ebene und entscheidet über SKU-Aufnahme, Substitutionen und Sortimentsprioritäten. Bei Rückrufen identifiziert diese Rolle das betroffene Lot/SKU und initiiert die Eskalationskette [4]. Bei Verpackungsänderungen ist sie die Erst-Signal-Rolle und koordiniert

Spezifikationsänderungen über Monate hinweg [4].

**Logistik / Distribution** überwacht die operative Lieferkette in Echtzeit und eskaliert an den Einkauf bei Lieferausfällen. Die Eskalationskette verläuft: Logistik → Einkauf → Sortimentsplanung mit einem Zeitrahmen von Stunden bis einem Tag [4]. Echtzeit-Kommunikation ermöglicht proaktive Information aller Stakeholder über potenzielle Verzögerungen [4].

**Prognose / Demand Forecasting** ist in die Supply-Chain-Planungssysteme integriert und berücksichtigt filial-spezifische Nachfrage, Promotions und Saisonalität [4]. Bei Nachfragespitzen ist diese Rolle die Erst-Signal-Rolle und erkennt Abweichungen über POS-Anomalien [4].

**Filialnachbestellung** arbeitet weitgehend automatisiert und löst Alarm aus bei OSA unter 95% für A-Artikel [4]. Bei IT-Ausfällen ist diese Rolle die Erst-Signal-Rolle, da das automatisierte Bestellsystem als erstes betroffen ist [4].

## 5 Systemzuordnung und Benachrichtigungsmodell

Die internen Dokumente referenzieren Systeme generisch als „Supply-Chain-Planungssysteme“, „Tracking-Systeme“ und „zentralisiertes Bestellsystem“ [4]. Für die Simulation werden diese auf konkrete Backend-Systeme gemappt:

| Rolle                     | Primärsystem                           | Sekundärsysteme                        | Benachrichtigungsmodus                                             |
|---------------------------|----------------------------------------|----------------------------------------|--------------------------------------------------------------------|
| Einkauf<br>(Markus Weber) | SAP Ariba (Verträge, Lieferantenmgmt.) | SAP ERP (Bestellungen), Outlook, Teams | Push: OTD-Alerts, Preisabweichungen; Pull: Vertragsstatus          |
| Sortimentsplanung         | SAP S/4HANA (Sortimentssteuerung)      | SAP ERP, Teams                         | Push: Rückruf-Alerts; Pull: Margenanalysen                         |
| Logistik                  | Manhattan WMS (Lagerverwaltung)        | SAP ERP (Transport), Teams             | Push: Lieferverzögerungen, Kapazitätsengpässe; Pull: Routenplanung |
| Prognose                  | SAP S/4HANA (Demand Planning)          | SAP ERP                                | Push: MAPE-Überschreitungen; Pull: Forecast-Modelle                |
| Filialnachbestellung      | SAP ERP (Auto-Bestellung)              | Manhattan WMS                          | Push: OSA-Alarme; Pull: Bestandsabfragen                           |
| Supply Chain Coordinator  | SAP S/4HANA, Teams                     | Outlook, Manhattan WMS                 | Push: Eskalationen; Pull: Status-Dashboards                        |

Tabelle 2: Systemzuordnung und Benachrichtigungsmodus pro Rolle

Das Benachrichtigungsmodell basiert auf schwellenwert-basierten Triggern, die als Push-Mechanismus fungieren [4]. Die Simulation nutzt definierte Schwellenwerte als Auslöser für Rollenaktivierung: Bestandsreichweite unter Mindestdeckungstage, Forecast-Error-Varianz über Toleranzband und Lieferzeit-Abweichung über definierte Schwelle [4]. Bei wiederholten Signalverletzungen werden progressiv höhere Eskalationsstufen ausgelöst [4].

## 6 Kommunikationsflüsse und Eskalationslogik

**Drei-Stufen-Eskalationsmodell.** Die Eskalationsstruktur bei ALDI SÜD folgt einem gestuften Modell [4]: Stufe 1 (Operative Erkennung) — die signalempfangende Rolle erkennt die Abweichung durch automatisierte Schwellenwert-Überwachung. Stufe 2 (Taktische Koordination) — die betroffene Rolle kontaktiert die nächste relevante Funktion. Stufe 3 (Strategische Entscheidung) — bei schwerwiegenden oder anhaltenden Störungen eskaliert die Koordination zu funktionsübergreifenden Risikorateen [4].

**Kanal-Routing für die Simulation.** Da die Quelldokumente keine spezifischen E-Mail- oder Teams-Routing-Regeln definieren [4], wird folgendes Routing-Schema für die Simulation spezifiziert: Microsoft Teams wird für Echtzeit-Kommunikation bei zeitkritischen Eskalationen verwendet (Stufe 1 und 2 mit Zeitrahmen unter 24 Stunden). Microsoft Outlook wird für formelle Benachrichtigungen, Dokumentation und Eskalationen mit längerem Zeitrahmen eingesetzt (Stufe 2 und 3). SAP-System-Alerts werden als In-App-Benachrichtigungen in den jeweiligen SAP-Modulen dargestellt.

**Timing der Benachrichtigungen.** Die konsolidierte Eskalationsmatrix zeigt unterschiedliche Zeitrahmen von der Erkennung bis zur Reaktion [4]: IT-Ausfall erfolgt in Minuten, Lieferausfall in Stunden bis einem Tag, Nachfragespitzen in Stunden, Extremwetter in Stunden bis Tagen, Rückruf innerhalb von 24 Stunden, Rohstoffpreisän-

derungen in Tagen bis Wochen, Mindestloohnerhöhungen in Wochen bis Monaten und Verpackungsänderungen über Monate.

**Demo-Modus: Sofortige Materialisierung.** Da in einem Demo-Szenario weder Tage noch Stunden gewartet werden kann, arbeitet die Simulation im „Pre-Aged“-Modus: Beim Auslösen eines Disruptions-Events werden alle Zwischenschritte der Eskalationskette so behandelt, als wäre die Störung bereits seit dem realistischen Zeitrahmen bekannt. Die Event-Timestamps werden rückdatiert (z.B. `anchor_date - detection_hours`), und die gesamte Nachrichtenkette – von der Erst-Erkennung über die Eskalation bis zur finalen Benachrichtigung beim Buyer – wird innerhalb weniger Sekunden vollständig generiert und in die Inbox geschrieben. Der Benutzer drückt den Knopf und sieht sofort das Endergebnis, als hätte der Prozess bereits stattgefunden.

**Wann erreicht Information den Buyer – interne und externe Absender.** Der Buyer erhält Benachrichtigungen sowohl von internen Kollegen als auch von externen Kontakten. Intern sendet die Logistik eine Teams-Nachricht wenn OTD unter 90% über eine Woche fällt [4], und die Sortimentsplanung kontaktiert ihn via Teams bei notwendigen Substitutionsentscheidungen. SAP Ariba generiert System-Alerts bei Vertragsabweichungen.

Zusätzlich simuliert das System externe Kommunikation: Lieferanten-Ansprechpartner (z.B. der Key-Account-Manager bei NewCoffee oder der Vertriebsleiter bei Storck) senden E-Mails bei Preiserhöhungsforderungen, Lieferunfähigkeit oder Vertragsänderungen. Darüber hinaus werden simulierte Branchen-Newsletter (z.B. Rohstoffpreis-Updates, Logistik-Marktberichte) und Hersteller-Rundschreiben in die Outlook-Inbox eingespeist, um ein realistisches E-Mail-Aufkommen mit gemischten internen und externen Absendern abzubilden.

## 7 Agenten-Architektur und Event-Simulation

Jede der fünf Supply-Chain-Rollen wird als autonomer Agent mit drei Schichten modelliert [4]: Die **Reasoning-Schicht** enthält die Geschäftsregeln und Entscheidungslogik (z.B. „Wenn Bestandsreichweite < 3 Tage UND kein Alternativlieferant verfügbar → Eskalation an Sortimentsplanung“). Die **Prozess-Schicht** bildet die Geschäftsprozesse ab, einschließlich Bestellauslösung, Routenplanung und Verhandlungsprotokoll. Die **Kommunikationsschicht** definiert die Interaktionsprotokolle zwischen Agenten [4].

Das Flowr Framework dient als zentrale Orchestrierungsschicht und koordiniert die Übergaben zwischen spezialisierten Agenten [4]. Die Zeitsynchronisation erfolgt durch sequenzielle Verhaltensänderungen der Agenten und resultierende Interaktionen in spezifizierten Agenten-Netzwerken [4].

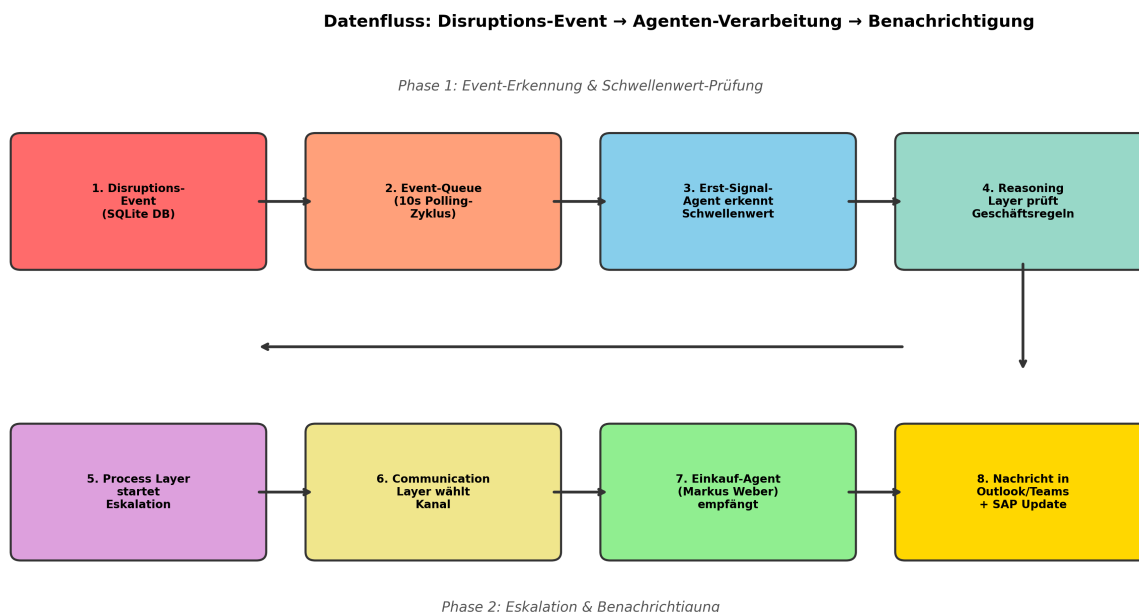


Figure 1: Datenfluss von Disruption zu Benachrichtigung

Diagrammerklärung: Das Diagramm zeigt den achtstufigen Datenfluss von einem Disruptions-Event in der SQLite-Datenbank über die Event-Queue mit 10-Sekunden-Polling, die Erkennung durch den Erst-Signal-Agenten, die Schwellenwert-Prüfung im Reasoning Layer, die Eskalationslogik im Process Layer, das Kanal-Routing im Communication Layer bis zur finalen Benachrichtigung bei Markus Weber in Outlook/Teams und dem SAP-Update [4].

**10-Sekunden-Polling-Zyklus.** Jeder Agent pollt im 10-Sekunden-Takt die Event-Queue in der SQLite-Datenbank. Der Polling-Zyklus umfasst folgende Schritte: Der Agent prüft die events-Tabelle auf neue, unverarbeitete Events mit Relevanz für seine Rolle. Bei Erkennung eines relevanten Events wird die Reasoning-Schicht aktiviert und prüft die Geschäftsregeln gegen die aktuellen KPI-Werte. Falls ein Schwellenwert überschritten ist, startet die Prozess-Schicht den definierten Eskalationsworkflow. Die Kommunikationsschicht generiert die entsprechenden Nachrichten und routet sie an die Ziel-Agenten oder Benutzer-Interfaces.

**Datenfluss-Beispiel: Lieferausfall bei NewCoffee.** Ein simuliertes Event „NewCoffee meldet Produktionsausfall“ wird in die events-Tabelle geschrieben. Der Logistik-Agent erkennt im nächsten Polling-Zyklus die ETA-Verschiebung und prüft: OTD fällt unter 90% → Schwellenwert verletzt. Der Process Layer des Logistik-Agenten initiiert Eskalation an den Einkauf-Agenten. Der Communication Layer generiert eine Teams-Nachricht an Markus Weber mit Betreff, betroffenen SKUs und empfohlenen Maßnahmen. Parallel wird ein SAP-Ariba-Alert für den Lieferantenvertrag erstellt.

## 8 Technische Architektur: Datenbank, Frontend und MCP-Server

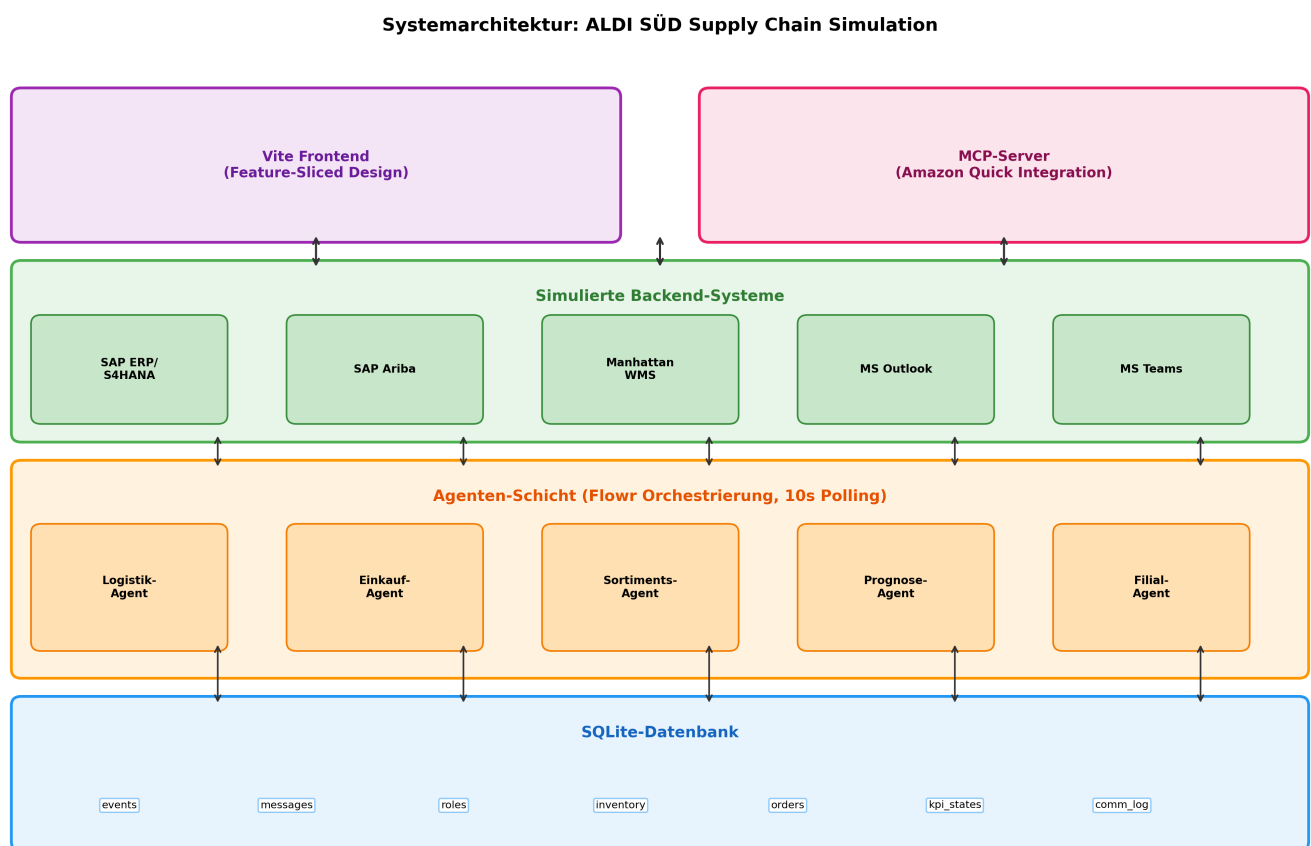


Figure 2: Systemarchitektur

Diagrammerklärung: Die Architektur zeigt vier Schichten — die SQLite-Datenbank als Persistenzschicht mit acht Kerntabellen, die Agenten-Schicht mit fünf rollenspezifischen Agenten unter Flowr-Orchestrierung, die simulierten Backend-Systeme (SAP ERP/S4HANA, SAP Ariba, Manhattan WMS, MS Outlook, MS Teams) und die Präsentationsschicht mit Vite Frontend und MCP-Server für Amazon Quick [4, 1, 2, 3].

### 8.1 SQLite-Datenbank-Schema

Die SQLite-Datenbank bildet den zentralen Datenspeicher des Simulationssystems. Das Schema umfasst folgende Kerntabellen:

**events** — Speichert alle Disruptions-Events mit Feldern: id (INTEGER PRIMARY KEY), event\_type (TEXT, z.B. 'supplier\_failure', 'price\_increase'), severity (TEXT: 'low' / 'medium' / 'high' / 'critical'), affected\_supplier (TEXT), affected\_skus (JSON-Array), created\_at (TIMESTAMP), status (TEXT: 'pending' / 'processing' / 'resolved'), scenario\_id (INTEGER FK), margin\_impact\_bps (REAL).

**roles** — Definiert die Agenten-Rollen: `id`, `name` (TEXT), `department` (TEXT), `org_level` (TEXT: 'holding' / 'regional' / 'automated' ), `primary_system` (TEXT), `polling_active` (BOOLEAN).

**messages** — Kommunikationslog: `id`, `from_role_id` (FK), `to_role_id` (FK), `channel` (TEXT: 'teams' / 'outlook' / 'sap'), `subject` (TEXT), `body` (TEXT), `priority` (TEXT), `created_at` (TIMESTAMP), `read_at` (TIMESTAMP NULL), `event_id` (FK).

**inventory** — Bestandsstatus: `id`, `sku_id` (TEXT), `supplier_id` (FK), `current_stock` (REAL), `coverage_days` (REAL), `reorder_point` (REAL), `last_updated` (TIMESTAMP).

**kpi\_states** — Aktuelle KPI-Werte pro Rolle: `id`, `role_id` (FK), `kpi_name` (TEXT), `current_value` (REAL), `threshold_value` (REAL), `status` (TEXT: 'normal' / 'warning' / 'critical' ), `last_updated` (TIMESTAMP).

**suppliers** — Lieferantenstammdaten: `id`, `name` (TEXT), `category` (TEXT), `contract_price` (REAL), `otd_current` (REAL), `risk_score` (REAL), `buyer_role_id` (FK).

**escalations** — Eskalationsprotokoll: `id`, `event_id` (FK), `from_role_id` (FK), `to_role_id` (FK), `escalation_level` (INTEGER: 1/2/3), `trigger_condition` (TEXT), `created_at` (TIMESTAMP), `resolved_at` (TIMESTAMP NULL).

**demo\_config** — Steuerungstabelle für den Demo-Modus: `id`, `anchor_date` (DATE, wird bei jedem Demo-Start auf das aktuelle Datum gesetzt), `active_user_name` (TEXT, aus Login aufgelöst), `active_user_email` (TEXT), `scenario_speed` (TEXT: 'instant' / 'realtime' ), `seed_version` (INTEGER).

## 8.2 Seed-Daten-Strategie mit relativen Datumswerten

Da die Demo unabhängig vom tatsächlichen Datum jederzeit gestartet werden können muss, verwenden alle Seed-Daten relative Zeitangaben bezogen auf ein `anchor_date`. Beim Start der Simulation wird `anchor_date` in der `demo_config`-Tabelle auf das aktuelle Datum gesetzt, und alle Seed-Timestamps werden als Offsets berechnet (z.B. `anchor_date - 2 days`, `anchor_date - 4 hours`). So entsteht eine konsistente historische Datenbasis, die niemals veraltet.

Die Seed-Daten umfassen: historische KPI-Verläufe der letzten 30 Tage (relativ zu `anchor_date`) für alle Rollen, Bestandsbewegungen und Lieferhistorie der letzten 14 Tage, bereits abgeschlossene Eskalationen als Referenzdaten, eine Auswahl vorausgefüllter Outlook-E-Mails und Teams-Nachrichten (interne Abstimmungen, externe Lieferanten-Korrespondenz, Newsletter), sowie Lieferanten-Performance-Daten mit realistischen OTD-Schwankungen. Alle diese Daten werden beim Initialisierungsskript (`seed.ts` oder `seed.py`) dynamisch generiert, sodass der Benutzer dem System auch vor Auslösung eines Disruptions-Events Fragen zu historischen Daten stellen kann.

## 8.3 Vite-Frontend-Architektur

Das Frontend nutzt Vite mit Feature-Sliced Design für saubere modulare Grenzen und sofortige Feedback-Loops [2]. Die Architektur verwendet native ES-Module und eine optimierte Rollup-basierte Build-Pipeline. Das Frontend wird in folgende Feature-Slices aufgeteilt:

Die **Simulations-Steuerung** (Route: `/simulation`) bietet Controls zum Auslösen von Disruptions-Events, Zeitraffer-Steuerung und Szenario-Auswahl. Die **E-Mail-Ansicht** (Route: `/outlook`) simuliert eine Outlook-Inbox für Markus Weber mit eingehenden Nachrichten, Prioritäts-Markierungen und Antwortmöglichkeiten. Die **Teams-Ansicht** (Route: `/teams`) zeigt Kanal-basierte Kommunikation mit Echtzeit-Nachrichten der anderen Agenten. Die **SAP-Ansicht** (Route: `/sap`) präsentiert Dashboards für Bestellstatus, Lieferanten-Performance und Vertragsübersichten aus SAP ERP, S/4HANA und Ariba. Die **Manhattan-WMS-Ansicht** (Route: `/wms`) zeigt Lagerbestände, Wareneingänge und Distributionsstatus.

Für die Verbindung zwischen Vite-Frontend und SQLite-Backend wird das Soul-Pattern empfohlen — ein automatischer SQLite RESTful und Realtime API Server [3]. Alternativ kann SQL.js für clientseitige Abfragen mit einem Vite+React+TypeScript-Boilerplate eingesetzt werden [6].

## 8.4 MCP-Server-Design für Amazon Quick

Das MCP-Protokoll folgt einer Host → Client → Server Architektur, wobei der Host (Amazon Quick) einen dedizierten MCP-Client pro Server erstellt [1]. Server exponieren Tools (strukturierte Operationen) und Resources (Daten wie Dateien/ Datenbanken) via JSON-RPC 2.0, wobei Clients Capabilities durch natürlichsprachliche Tool-Beschreibungen entdecken [1].

Die Authentifizierung basiert auf OAuth 2.1 mit Dynamic Client Registration, PKCE (S256) und Resource Indicators (RFC 8707) [7]. Amazon Quick sichert MCP-Verbindungen mittels PKCE mit S256 und RFC 8707, um Access

Tokens an spezifische MCP-Server zu binden [8]. Amazon Quick enthält einen integrierten MCP-Client, der sich mit Remote-MCP-Servern verbindet, exponierte Tools entdeckt und sie als aufrufbare Aktionen während KI-Assistenten-Konversationen registriert [9].

Für die Simulation werden folgende MCP-Tools definiert: `get_supply_chain_status` (liefert aktuelle KPI-Werte und offene Events), `get_supplier_performance` (Lieferanten-Metriken für das Buyer-Portfolio), `get_pending_messages` (ungelesene Nachrichten aus Outlook/ Teams für den angemeldeten Benutzer), `trigger_disruption_event` (startet ein neues Szenario), `get_escalation_history` (zeigt Eskalationsverläufe), `get_inventory_coverage` (Bestandsreichweiten pro SKU) und `get_user_identity` (liefert den aktuellen Benutzernamen und die Rolle aus `demo_config`).

Entscheidend für die Amazon-Quick-Integration ist, dass der MCP-Server „Outlook“ sich gegenüber Quick als E-Mail-System identifiziert. Die Tool-Beschreibungen müssen natürlichsprachlich klar formuliert sein (z.B. „Liest E-Mails aus dem Outlook-Postfach des angemeldeten Benutzers“ ), damit Quick den Server korrekt als E-Mail-Kanal erkennt und entsprechende Benutzeranfragen dorthin routet [1, 9]. Der Benutzername wird aus dem OAuth-Token des Logins extrahiert und in `demo_config.active_user_name` persistiert, sodass alle MCP-Server den Benutzer konsistent unter seinem tatsächlichen Namen adressieren.

## 9 Disruptionsszenarien und Propagationslogik

Das Simulationssystem implementiert acht Disruptionsszenarien mit unterschiedlichen Wahrscheinlichkeiten, Dauern und Margen-Impacts [4]:

| Szenario                  | Margin-Impact (bps) | Wahrscheinlichkeit p.a. | Dauer       | Erst-Signal-Rolle          | Zeitraumen Erkennung  |
|---------------------------|---------------------|-------------------------|-------------|----------------------------|-----------------------|
| Rückruf (Recall)          | 18 (5-30)           | 20%                     | 1-4 Wochen  | Sortimentsplanung/ Einkauf | 24 Stunden [4]        |
| Verpackung 18 (Packaging) |                     | 100%                    | Monate      | Sortimentsplanung          | Monate [4]            |
| Saisonspitzen 22 (15-30)  |                     | 100%                    | 4-6 Wochen  | Prognose                   | Stunden [4]           |
| IT-Ausfall                | 25 (10-40)          | 8%                      | 1-7 Tage    | Filialnachbestellung       | Minuten [4]           |
| Extremwetter 30 (10-50)   |                     | 25%                     | 2-12 Wochen | Prognose                   | Stunden bis Tage [4]  |
| Mindestlohn 45 (30-60)    |                     | 100%                    | Permanenter | Einkauf                    | Wochen bis Monate [4] |
| Schifffahrt (Shipping)    | 50 (20-80)          | 12%                     | 2-8 Wochen  | Logistik                   | Stunden bis 1 Tag [4] |
| Pandemie                  | 125                 | 4%                      | Monate+     | Multi-Signal               | Stunden [4]           |

Tabelle 3: Disruptionsszenarien mit Parametern für die Simulation [4]

**Korrelierte Risikocluster.** Die Simulation modelliert zwei korrelierte Risikocluster: „Geopolitik/Energie/Transport“ und „Klima/Agrar/Frischware“, die simultane Aktivierung mehrerer Eskalationspfade verursachen [4]. Bei Aktivierung eines Clusters werden mehrere Events gleichzeitig in die Event-Queue geschrieben.

**Eskalationsketten pro Szenario [4].** Beim Lieferausfall verläuft die Kette: Logistik (Verzögerung erkannt) → Einkauf (Dual-Sourcing aktiviert) → Sortimentsplanung (Substitution geprüft). Bei Nachfragespitzen: Prognose (Abweichung erkannt) → Filialnachbestellung (POS-Anomalie bestätigt) → Logistik (Kapazität erhöht) → Einkauf (Volumenabruf). Beim Produktrückruf: Sortimentsplanung (Lot/SKU identifiziert) → Einkauf (Lieferant kontaktiert) → Logistik (Distribution eingefroren) → Filialnachbestellung (Ware separiert) → Prognose (Nachfrage neu berechnet) [4].

**Relevanz für Markus Webers Portfolio.** Bei einem Lieferausfall von NewCoffee (Kaffee-Lieferant) sind die SKUs Markus-Kaffee und Barissimo direkt betroffen. Der Logistik-Agent erkennt die OTD-Verschlechterung, eskaliert an den Einkauf-Agenten, und Markus Weber erhält eine Teams-Nachricht mit der Aufforderung, Dual-Sourcing zu prüfen oder Alternativlieferanten zu aktivieren. Bei Mindestloohnerhöhungen erkennt Markus Weber als Erst-Signal-Rolle die Preisforderungen seiner Lieferanten und muss entscheiden, ob die Kostenüberschreitung über 5% liegt und eine Eskalation an die Sortimentsplanung erforderlich ist [4].

## 10 Implementierungsempfehlungen für den Coding Agent

---

**Technologie-Stack.** Das System verwendet: SQLite 3 als Datenbank, Node.js mit dem Soul-Framework als REST-API-Layer [3], Vite + React + TypeScript für das Frontend [2, 6], einen MCP-Server in Node.js/Python mit JSON-RPC 2.0 [1], und einen Agent-Orchestrator basierend auf dem Flowr-Pattern [4].

**Implementierungsreihenfolge.** Die empfohlene Reihenfolge priorisiert die MCP-Server-Integration, um frühzeitig validieren zu können, ob Amazon Quick den „Outlook MCP“ tatsächlich als E-Mail-System erkennt: (1) MCP-Server implementieren mit Tool-Definitionen, natürlichsprachlichen Beschreibungen und OAuth-2.1-Authentifizierung — insbesondere den Outlook-MCP-Server mit klarer Selbstidentifikation als E-Mail-System [1, 9]. (2) SQLite-Datenbank-Schema erstellen und mit relativ-datierten Seed-Daten befüllen (Rollen, Lieferanten, SKUs, historische KPI-Werte, vorausgefüllte Nachrichten); das Seed-Skript berechnet alle Timestamps relativ zu `anchor_date`, das bei jedem Demo-Start auf das aktuelle Datum gesetzt wird. (3) Agent-Engine implementieren mit 10-Sekunden-Polling-Loop, Reasoning-Regeln, Eskalationslogik und Demo-Modus (Pre-Aged-Events für sofortige Materialisierung). (4) REST-API-Layer aufsetzen für Frontend-Zugriff auf Datenbank. (5) Vite-Frontend mit Feature-Sliced Design und den fünf Views (Simulation, Outlook, Teams, SAP, Manhattan). (6) Szenario-Engine für die acht Disruptionsszenarien mit konfigurierbaren Parametern.

**Konfigurierbarkeit.** Alle Schwellenwerte (OTD < 90%, Kosten > 5%, OSA < 95%, MAPE > 20%) [4] sollen als konfigurierbare Parameter in einer `config`-Tabelle oder JSON-Datei gespeichert werden. Die Szenario-Parameter (Wahrscheinlichkeit, Dauer, Margin-Impact) [4] sollen ebenfalls konfigurierbar sein, um verschiedene Simulationläufe zu ermöglichen.

**Testszenarien.** Für die Validierung werden drei Kernszenarien empfohlen: (1) Lieferausfall NewCoffee — testet die Kette Logistik → Einkauf → Sortimentsplanung mit sofortiger Teams-Benachrichtigung an den Buyer (Pre-Aged-Modus). (2) Mindestloohnerhöhung — testet den Buyer als Erst-Signal-Rolle mit simulierter externer E-Mail-Korrespondenz (Lieferanten-Ansprechpartner und Newsletter). (3) Korreliertes Cluster „Geopolitik/Energie/Transport“ [4] — testet simultane Aktivierung mehrerer Agenten und parallele Eskalationspfade.

## 11 References

---

1. <https://modelcontextprotocol.io/docs/learn/architecture>
2. <https://feature-sliced.design/blog/vite-build-tool-architecture>
3. <https://github.com/thevahidal/soul>
4. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/379a5086-18ca-4601-9b3e-d7d859a2af5c?documentId=b7a41b18e-4ed8-ba24-3076682c106e>
5. <https://us-east-1.quicksight.aws.amazon.com/sn/spaces/379a5086-18ca-4601-9b3e-d7d859a2af5c?documentId=d80b50af3-4bba-ad08-95cf251e7f14>
6. <https://github.com/iliareshetov/react-sqlite-boilerplate>
7. <https://fast.io/resources/mcp-server-authentication/>
8. <https://docs.aws.amazon.com/quick/latest/userguide/mcp-integration.html>
9. <https://aws.amazon.com/tw/blogs/machine-learning/integrate-external-tools-with-amazon-quick-agents-using-model-context-protocol-mcp/>